

Campus Connect: Website Guide

CS 5910 System Design and Implementation · SUNY Old Westbury · Fall 2026

Team: Pavel Clarke (Team Leader), Julio Larrea, Muiz M. Onifade · October 7, 2026

This guide explains every page of the Campus Connect website: what the user sees (frontend), what the server does (backend), and which database tables each page reads or changes. It ends with an overview of the three layers across the whole site.

Contents

1. How the site is built
2. Pages before signing in (Sign in, Sign up, Account created, Password reset)
3. The dashboard layout
4. Student pages
5. Faculty pages
6. Admin pages
7. Statistics Department pages
8. Pages shared by several roles
9. Overview: frontend, backend and database
10. How the three layers connect

1. How the site is built

Every page works the same way underneath:

- **It is a single-page website.** The browser loads one HTML page (`public/index.html`) once. After that, JavaScript (`public/login.js`) swaps the content on screen without loading new pages.
- **Every page asks the server for its data.** When a sidebar link is clicked, the browser calls the server (for example `/api/views/transcript`). The server runs SQL queries against MySQL and returns the page's content as data: a title, stat cards, filters, tables and forms. The browser draws it with one shared set of styles.
- **Every change goes through the server.** Buttons such as Add, Drop, Create section, Create account and Reset password send a request to the server. The server checks the rules, writes to MySQL, and replies with a success or error message.

For each page below: **Frontend** is what you see and click, **Backend** is the server route and logic, and **Database** is which MySQL tables are read or written.

2. Pages before signing in

Page 1. Sign in

FRONTEND

- Two-panel layout: a blue welcome panel on the left and the sign-in form on the right.
- Fields for **Email or ID** and **Password**; one-click test-account buttons (Student, Faculty, Admin, Stat-Dept); a "Create an account" link.
- Errors appear in a red box. After 3 failed tries the box also shows a "Forgot your password? Reset it" link.

BACKEND

- `POST /api/login` . If the input is all digits it looks up the user ID, otherwise the email. A locked account is refused immediately.
- The typed password is hashed with the stored salt (scrypt) and compared to the stored hash; plain-text passwords are never stored or compared.
- Wrong password: the failed-attempt count goes up by 1. At 3 the server tells the page to offer a reset; at 5 the account locks.
- Right password: the count resets to 0 and a **session** starts (a secure cookie that keeps you signed in for 8 hours). `GET /api/me` restores the session after a refresh; `POST /api/logout` ends it.

DATABASE

- **Reads:** `Login` (email, password hash, attempt count, lock flag) and `User` (name, role).
- **Writes:** `Login.no_Of_Tries` and `Login.lock_Var` .

Page 2. Create a student account (sign up)

FRONTEND

- Fields: first, middle (optional) and last name; gender; date of birth; full-time or part-time; major; optional address (street, city, state, ZIP); password and confirmation.
- The major list is filled from the database (`GET /api/majors`). Fields sit in three columns on a computer and one column on a phone.

BACKEND

- `POST /api/signup` (`server/accounts.js`).
- Validation: names are letters; age between 14 and 100; state 2 letters and ZIP 5 digits if given; password 8+ characters with a letter and a number, and both copies match; the major is an undergraduate major.
- Generates a **random unused 6-digit student ID** (100000–199999) and a **campus email** from the name (adding 2, 3, ... if taken).
- Assigns the full-time professor in the major's department with the fewest advisees (never more than 15).
- All records are written in **one transaction**: if any one fails, none are saved.

DATABASE

Writes 7 rows:

Table	What is stored
User	Name, gender, date of birth, address, user type
Login	Email and hashed password
Student	Year (Freshman) and type (Undergraduate)
Undergraduate_Student	Department and full-time or part-time
Full_Time_Undergraduate or Part_Time_Undergraduate	Credit limits (12–16 or 1–8)
Student_Major	The chosen major
Advisor	The assigned advisor

Page 3. Account created

FRONTEND

Cards showing the new **Student ID, Campus email, Name, Major and Advisor**, and a **Sign in now** button that opens sign-in with the email filled in.

BACKEND

None of its own; it displays what the sign-up request returned.

DATABASE

None of its own.

Page 4. Reset your password

- FRONTEND** Reached from the reset link. Fields: email or ID, student/user ID, date of birth, new password and confirmation. On success it returns to sign-in with a green confirmation message.
- BACKEND**
- `POST /api/reset-password` . Allowed only after 3+ failed sign-ins, or on a locked account.
 - Verification: the ID and date of birth entered must match the account. After 5 wrong verification attempts, resets for that account are blocked for 15 minutes.
 - On success: saves the new password as a fresh hash, resets failed attempts to 0, and unlocks the account.
- DATABASE**
- **Reads:** `Login` and `User.DOB` .
 - **Writes:** `Login.user_Password` , `no_Of_Tries` and `lock_Var` .

3. The dashboard layout (shared by every signed-in page)

- FRONTEND**
- **Sidebar:** a dark blue bar listing only the pages the user's role may see, with name, email and a Sign out button at the bottom.
 - **Top bar:** the role badge and ID. **Content:** a title, a subtitle, then the page's content.
 - On phones the sidebar becomes a top menu.
- BACKEND** `GET /api/views` returns the pages for the user's role (defined in `server/views.js`). Every page request checks the role **on the server**, so typing another role's page address returns "That page is not available for your role."
- DATABASE** None directly; this is the frame the other pages load into.

4. Student pages

Page 5. Overview

- FRONTEND**
- "Welcome, [name]" with level, year, full/part-time and academic standing.
 - Cards: Major, Minor, GPA, Credits earned, this semester's classes, Active holds. A table of this semester's schedule.
- BACKEND**
- The current semester is the most recent semester whose start date has passed.
 - GPA: each grade is converted to points (A = 4.0, A- = 3.7 ... F = 0) and weighted by course credits.
- DATABASE** **Reads:** `Student` , `Undergraduate_Student` / `Graduate_Student` , `Full_Time_...` / `Part_Time_...` , `Student_Major` + `Major` , `Student_Minor` + `Minor` , `Student_History` , `Enrollment` , `Course_Section` , `Course` , `Student_Hold` , and the scheduling tables.

Page 6. My schedule

- FRONTEND** Two tables: the **current semester** and the **next semester**. Columns: CRN, course, title, section, credits, days, time, room, instructor.
- BACKEND** One shared section query turns raw IDs into readable text: time slot 3 becomes "MW 11:00 AM – 12:20 PM", room NSC-104 becomes "Natural Sciences 104".
- DATABASE** **Reads:** Enrollment → Course_Section → Course, Time_Slot_Day, Time_Slot_Period → Period, Room → Building, User (instructor name).

Page 7. Transcript

- FRONTEND** Cards for cumulative GPA, credits earned and courses completed; a table of every completed course in order (semester, course, title, credits, grade).
- BACKEND** GPA and earned credits are calculated in SQL. Failed courses (F) earn no credit.
- DATABASE** **Reads:** Student_History → Course_Section → Course, Semester.

Page 8. Holds & advisors

- FRONTEND** **Active holds** table (type, date placed, with a note that a hold blocks registration) and **My advisors** table (name, rank, department, email, office).
- BACKEND** Two read queries.
- DATABASE**
- **Holds:** Student_Hold + Hold.
 - **Advisors:** Advisor + Faculty + User + Login + Office + Faculty_Department + Department.

Page 9. Register (add and drop classes)

FRONTEND

- Filters: semester (only semesters with registration open) and department.
- Cards: registered credits (for example "3 of 16"), active holds, add period, drop period.
- **My registration:** the student's sections, each with a red **Drop** button.
- **Sections you can add:** each with a blue **Add** button.
- **Sections you can't add:** each with the reason (for example "You already passed CS110") and a **Try** button that sends the request anyway, to show it being refused.
- After an action the page reloads with a green **Done** or red **Not allowed** message.

BACKEND

POST /api/actions/add-section (server/actions.js). One function checks every registration rule in order:

1. The add period is open.
 2. The student has no active holds.
 3. The course level matches the student's (undergraduate or graduate).
 4. The student is not already registered for or taking that course.
 5. The student has not already passed it.
 6. Every prerequisite is passed with the minimum grade.
 7. A seat is available (10 max).
 8. The student stays under the credit limit (16 full-time, 8 part-time).
 9. It does not clash with the student's schedule (same day and same period).
- The same function sorts the lists into "can add" and "can't add", so the page and the button never disagree.
 - The add runs in a transaction that **locks the section's row**, so two students can't take the last seat at the same moment.
 - POST /api/actions/drop-section checks the drop period and that the student is enrolled in an ungraded section.

DATABASE

- **Reads:** Semester (add/drop dates), Student_Hold, Student, Student_History, Course_Prerequisite, Enrollment (seats, credits, time clashes), Time_Slot_Day / Time_Slot_Period.
- **Writes:** INSERT INTO Enrollment to add, DELETE FROM Enrollment to drop.

5. Faculty pages

Page 10. My teaching

FRONTEND

- Cards: rank, full/part-time, departments (with % time), office, sections this semester, advisees.
- Tables: this semester's teaching schedule with open seats; a **roster for each section** (student ID, name, year, email, **attendance %**); next semester's teaching schedule.

BACKEND

Attendance % is calculated in SQL as the share of class meetings marked "Present".

DATABASE

Reads: Faculty, Office, Faculty_Department + Department, Advisor (Count), Course_Section, Enrollment, User, Student, Login, Attendance.

Page 11. Advisees

FRONTEND	A table of the faculty member's advisees: ID, name, level, year, major, any holds, advising start date.
BACKEND	One read query, limited to the faculty member's own advisees.
DATABASE	Reads: <code>Advisor</code> , <code>Student</code> , <code>User</code> , <code>Student_Major</code> + <code>Major</code> , <code>Student_Hold</code> + <code>Hold</code> .

6. Admin pages

Page 12. Overview

FRONTEND	• Welcome with the admin's security level (Read-Write or Read-Only). • Cards: totals for students, faculty, departments, courses, this semester's sections and enrollments, students with holds, locked accounts. • Departments table: chair, manager, phone, email, number of faculty, number of courses.
BACKEND	Counting queries.
DATABASE	Reads: <code>Admin</code> , <code>Student</code> , <code>Faculty</code> , <code>Department</code> , <code>Course</code> , <code>Course_Section</code> , <code>Enrollment</code> , <code>Student_Hold</code> , <code>Login</code> , <code>Faculty_Department</code> , <code>User</code> .

Page 13. Student holds

FRONTEND	Every active hold, newest first: student ID, name, hold type, date.
BACKEND	One read query.
DATABASE	Reads: <code>Student_Hold</code> , <code>Hold</code> , <code>User</code> .

Page 14. Audit log

FRONTEND	Every admin change to user information: date and time, which admin, which user, action, field, old value, new value.
BACKEND	Joins the <code>User</code> table twice: once for the admin's name, once for the affected user's name.
DATABASE	Reads: <code>Audit_Log</code> , <code>User</code> .

Page 15. Create section

FRONTEND

- Form: semester (upcoming only), course, faculty, time slot, room, max seats.
- Each faculty label shows departments, type and current load, for example "(CS, Full-time, teaching 1 of 2)".
- Below the form, a "Newest Spring 2027 sections" list, where a new section appears at the top.

BACKEND

POST `/api/actions/create-section`. In one transaction it checks:

- The admin is **Read-Write**.
- Seats are 1–10 and fit the room's capacity.
- The semester has not started.
- The faculty member belongs to the course's department.
- Teaching load: full-time at most 2 sections, part-time at most 1.
- The faculty member is not already teaching at that time, and the room is not already in use at that time.

It then assigns the next CRN and section number.

DATABASE

- **Reads:** `Admin`, `Semester`, `Course`, `Faculty` + `User`, `Room`, `Faculty_Department`, `Course_Section`, the time-slot tables.
- **Writes:** `INSERT INTO Course_Section`.

7. Statistics Department pages

Page 16. Statistics

FRONTEND

- Cards: undergraduate and graduate counts, full-time and part-time faculty, this semester's sections, seats filled out of total.
- Tables: students per major; **grades by department** (count, average grade points, and the number of A, B, C, D and F grades).
- No student is ever named.

BACKEND

Aggregate queries only (`COUNT`, `SUM`, `AVG`); names and IDs are never selected.

DATABASE

Reads: `Student`, `Faculty`, `Course_Section`, `Enrollment`, `Major`, `Student_Major`, `Student_History`, `Course`, `Department`.

Page 17. Enrollment counts

FRONTEND

Semester and department filters; a table per section with registered count, max seats and available seats. No names.

BACKEND

One aggregate query, filtered by semester and department.

DATABASE

Reads: `Course_Section`, `Course`, `Enrollment` (counted, never listed).

8. Pages shared by several roles

Page 18. Master schedule (students, faculty, admins)

FRONTEND	Semester and department filters; every section with days, time, room, instructor and open seats.
BACKEND	The same shared section query as My schedule, filtered. Open seats = max seats minus enrollments, calculated live.
DATABASE	Reads: <code>Course_Section</code> , <code>Course</code> , the time-slot tables, <code>Room</code> , <code>Building</code> , <code>User</code> , <code>Enrollment</code> .

Page 19. Course catalog (all four roles)

FRONTEND	• Department filter. • Courses table: ID, title, credits, level, prerequisites with minimum grades. • Majors and minors table: each program's required courses.
BACKEND	Two read queries, filtered by department.
DATABASE	Reads: <code>Department</code> , <code>Course</code> , <code>Course_Prerequisite</code> , <code>Major</code> + <code>Major_Course_Requirement</code> , <code>Minor</code> + <code>Minor_Course_Requirement</code> .

9. Overview: frontend, backend and database

Frontend: what the user sees

- **Files:** `public/index.html` (page structure), `public/styles.css` (look and phone layout), `public/login.js` (all behavior).
- **How it works:** a single-page app in plain HTML, CSS and JavaScript, with no framework. One renderer draws every page from four building blocks: **stat cards, filters, tables (optionally with action buttons) and forms**. That is why all 19 pages look consistent.
- **What it never does:** decide anything that matters. It does not check passwords, enforce rules or calculate GPA. It shows what the server sends and passes clicks to the server.

Backend: the rules

- **Technology: Node.js with the Express framework**, in four files:
 - `index.js` : routes, sign-in and sessions.
 - `views.js` : the read-only pages and their SQL.
 - `actions.js` : add, drop and create section, with all rule checks.
 - `accounts.js` : sign-up and password reset.
- **Security:**
 - Role checks on every request: a student can't open admin pages or run admin actions, even by typing the address.
 - Passwords are stored as salted script hashes, never as plain text.
 - Every SQL query uses `?` placeholders (parameterized queries), which blocks SQL injection.
 - The session cookie is HTTP-only, so scripts on the page can't read it.
 - Multi-step changes run in transactions (all-or-nothing), and row locks prevent two people taking the same seat.
- **Requirement enforcement:** every SRS rule the site covers is enforced on the server, not in the browser.

Database: where everything lives

- **Technology: MySQL 8**, database `campus_connect`, with **40 tables** matching Section e of the System Manual.
- **Integrity:** primary keys, foreign keys, CHECK rules, and unique keys that stop a room or faculty member from being booked twice at the same time.
- **Data:** generated by `db/generate-data.js`, about 12,800 rows that follow the SRS rules: 305 users, 6 departments, 90 courses, 333 sections, over 2,000 enrollments and about 5,700 attendance records.
- **Setup and checks:** `npm run db:setup` (or `deploy/reset-db.sh` on the server) rebuilds it; `db/validate.sql` checks 20 rules against the loaded data.
- **How the site uses it:** 17 of the 19 pages read from MySQL every time they load (the "Account created" screen and the dashboard frame do not query it themselves). Writes happen on sign-in attempts, sign-up, password reset, add, drop and create section.

10. How the three layers connect

Your browser	--HTTPS-->	nginx (port 80/443)	-->	Node.js / Express (port 3000)	-->	MySQL (port 3306)
frontend		web server on		backend: checks role and rules,		database: stores
(HTML/CSS/JS)		Amazon Lightsail		runs SQL in transactions		all 40 tables

Example: the full path of one click. When the demo student (Julio Larrea) clicks **Add** on CS455:

1. The browser sends the CRN to `/api/actions/add-section`.
2. Express confirms he is a signed-in student.
3. It locks the section row and runs the nine registration checks against MySQL.
4. It inserts a row into `Enrollment` and commits.
5. It replies "Added CS455...".
6. The browser reloads the Register page, which now shows CS455 under his registration, read fresh from MySQL.

Hosting. All three layers run on one Amazon Lightsail Ubuntu server. nginx handles web traffic, systemd keeps the Node app running and restarts it after crashes or reboots, and MySQL runs on the same machine.